BCA MAJOR PROJECT / IMPLEMENTATION REPORT

CareerLens

AI Resume Analyzer and Job Recommendation System

Prepared for Saksham Jadhav | 28 September 2026

Abstract

CareerLens is an NLP-based career guidance application that extracts technical skills from PDF and DOCX resumes, compares them with curated role profiles, and produces an explainable match score, skill-gap analysis and learning plan. The application uses a 110-skill dictionary, 13 role profiles, boundary-aware alias matching and a hybrid of coverage and TF-IDF cosine similarity. A Streamlit dashboard presents rankings, evidence labels, charts, section-based feedback and custom job-description comparisons. PDF reports are generated in memory. The design avoids paid APIs and permanent storage of resumes. It supports the seven modules in the supplied specification while distinguishing documented keyword alignment from ability or hiring likelihood.

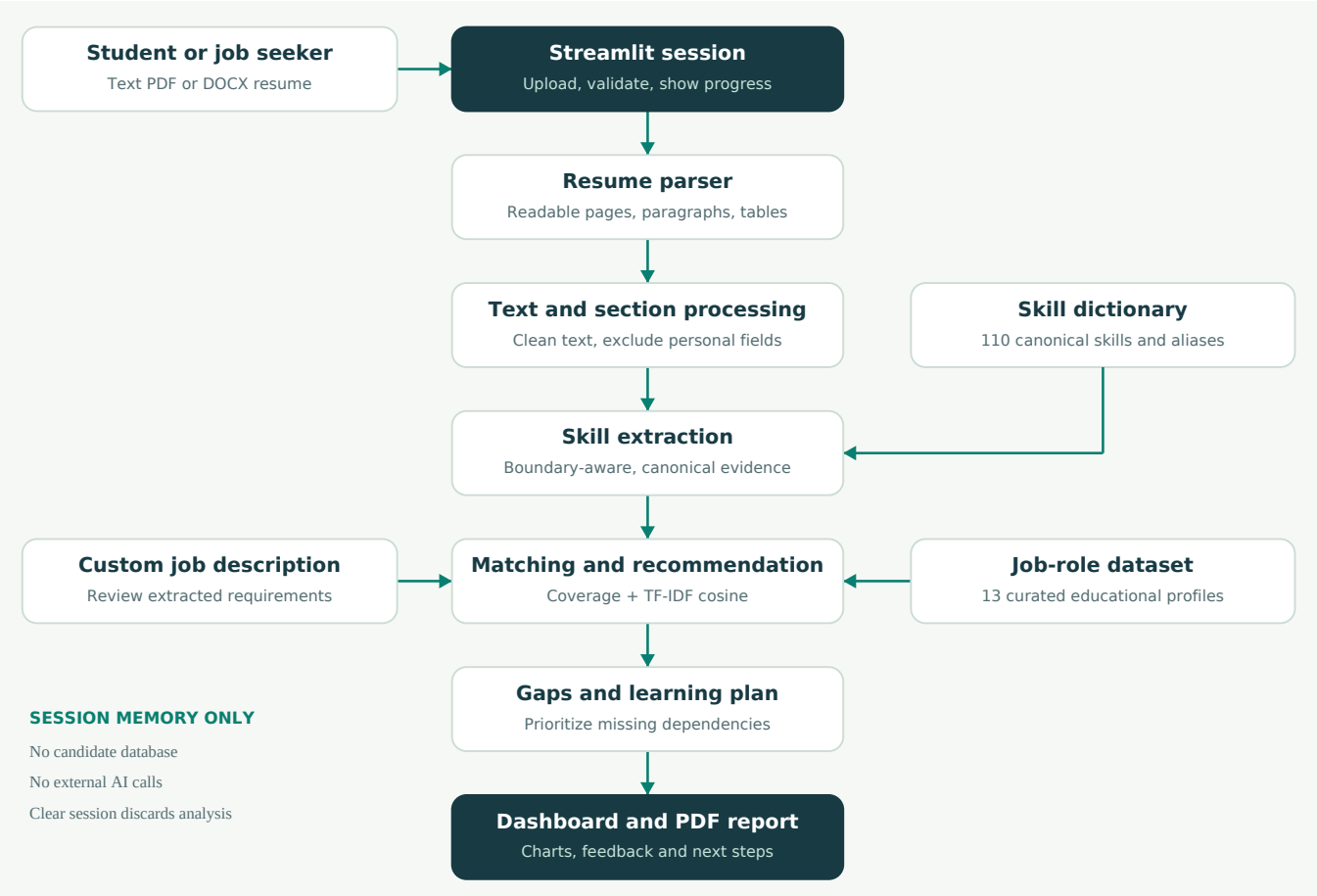
Problem statement and objectives

Students may have useful experience but struggle to communicate it in terms relevant to a target role. Generic keyword tools can conceal their scoring rules and treat missing words as missing ability. This project makes the extracted evidence visible, shows required and preferred gaps separately, explains every score component, and orders learning topics around prerequisites and practical projects.

Existing limitations and proposed system

Manual comparisons are slow and inconsistent. Simple substring searches confuse related words, while opaque generated scores are difficult to verify. CareerLens uses a controlled, reviewable dataset and deterministic rules. It preserves technical punctuation, validates uploaded documents, rejects unreadable inputs with useful messages, and offers an interactive dashboard without requiring an external backend.

Architecture and workflow



Raw documents and derived results remain in session memory. Public datasets and the fitted vectorizer are shared immutable resources. Resume feedback uses the extracted sections; the numerical score uses technical skill presence only.

Seven implemented modules

Module	Implemented behavior
1. Resume upload	PDF/DOCX, validation, file details and replace/remove controls
2. Extraction and cleaning	All readable pages, DOCX paragraphs/tables, preserved technical symbols
3. Skill extraction	Canonical aliases, boundaries, categories and context labels
4. Job-role dataset	13 curated profiles and 110 skills with prerequisites
5. Matching	Coverage, TF-IDF cosine, deterministic ranking and top three roles
6. Skill gaps	Required/preferred gaps and prerequisite-first learning plan
7. Dashboard	Plotly charts, custom JD, feedback and PDF report download

Matching algorithm

First, aliases map to canonical skill names. Each skill becomes a binary token, so repetitions do not add weight. TF-IDF is fitted once on the 13 role profiles. For each target, the resume vector is restricted to skills in that target's core or preferred set. Cosine similarity measures the overlap between this projected vector and the role vector.

Score = 75 x core coverage + 15 x preferred coverage + 10 x cosine similarity. Each input is between zero and one. A target without preferred skills uses 90 x core coverage + 10 x similarity. Scores are bounded to 0-100 and rounded to one decimal.

Because the projected resume vector contains a subset of the role's identically weighted terms, cosine equals the square root of the covered proportion of squared IDF weights. Adding a relevant skill cannot lower coverage or similarity. Unrelated skills and repeated terms have no effect. This is lexical matching, not semantic comprehension or supervised hiring prediction.

Skill gaps, learning plan and feedback

A gap means a required term was not detected, not that the candidate lacks the skill. Missing core topics are ordered by learning stage, with missing prerequisites placed first. Already-detected topics are skipped and the plan ends with a role-specific project. Preferred skills are optional additions. Contextual mentions in projects or work are distinguished from bare mentions without asserting proficiency. Feedback highlights missing sections, brief project descriptions and opportunities for truthful measurable outcomes.

Technology and deployment design

Part	Technology
Parsing	pypdf and python-docx
NLP and matching	Python regular expressions and scikit-learn TF-IDF/cosine
Interface and charts	Streamlit and Plotly
Reports and testing	ReportLab, Pytest, Ruff and Playwright acceptance checks
Hosting preparation	Streamlit Community Cloud, Render configuration and Docker

Python 3.12 is the tested runtime. No API key, database or model download is required. Launchers, pinned dependencies, a transitive lock file, a non-root Dockerfile and deployment instructions are included.

Testing and observed results

The evaluation uses 3 manually labelled fictional profiles exported to PDF and DOCX. All 6 documents preserved the expected normalized text and ranked the expected role first. Micro precision and recall for the explicitly labelled skill sets were both 1.00. These are small synthetic fixtures authored for this project, not an independent benchmark or estimate of real-world accuracy.

Profile and format	Expected and actual top role	Score
data_analyst.pdf	Data Analyst	95.5%
data_analyst.docx	Data Analyst	95.5%
ml_engineer.pdf	Machine Learning Engineer	92.7%
ml_engineer.docx	Machine Learning Engineer	92.7%
nlp_engineer.pdf	NLP Engineer	100.0%
nlp_engineer.docx	NLP Engineer	100.0%

Automated checks cover extraction, bad files, aliases, false positives, score bounds and monotonicity, protected-field counterfactuals, roadmaps, reports and Streamlit flows. The exact automated/browser counts and execution limitations are recorded in docs/VERIFICATION.md and the machine-readable reports.

Privacy and responsible AI

Gender, age, religion, nationality, photographs, marital status and disability are not scoring features. Recognized personal fields and sections are excluded before matching. The app makes no hiring or rejection decisions and states that absent keywords do not establish absent ability. Uploads and outputs are held in session memory, with no candidate database or external AI calls. The clear-session control discards application references; this is not a secure-memory-erasure guarantee.

Limitations, future scope and conclusion

The supported scope is readable English technical resumes. Scanned images require OCR, complex layouts may change reading order, and a controlled dictionary cannot infer every synonym or skill level. Curated role requirements vary from real vacancies. Future work should begin with consented, independently labelled evaluation, reviewed aliases, isolated OCR and measured semantic retrieval. Within its stated scope, CareerLens provides an inspectable workflow from resume upload to role comparison, skill gaps, practical next steps and a downloadable report.

References: supplied project specification; official Streamlit, pypdf, python-docx, scikit-learn and Docker documentation. Full links and technical explanation are in README.md.